Curso de Bash Scripting

1 - El intérprete de comandos

El intérprete de comandos actúa como una interfaz alfanumérica de entrada/salida entre el usuario y el Sistema Operativo (SO), siendo a su vez un programa informático que hace posible que el SO responda a las ordenes introducidas por el usuario mediante una serie de comandos. Su uso nos da una gran flexibilidad sobre el sistema, fundamental para todo aquel que quiera conocer mejor la esencia de los sistemas ***NIX** (Linux y UNIX) o quiera profundizar en el área de administración de sistemas, programación, redes y servidores. El intérprete de comandos también es llamado *intérprete de línea de comandos* (CLI en sus siglas en Inglés).

Desde su creación, en los entornos Unix/Linux la herramienta que más ha ayudado al administrador a realizar cualquier tipo de tarea administrativa siempre ha sido el intérprete de comandos o shell, incluso tras la implantación de servidores gráficos. Existen muchos intérpretes diferentes, cada cual con sus particularidades, pero el más ampliamente extendido hoy en día en la mayor parte de distribuciones es, sin duda, el intérprete Bash (Bourne Again Shell). Cronológicamente, los diferentes intérpretes *más usados* fueron apareciendo así:

Bourne Shell (sh)

Creador original: Stephen Bourne, de los Laboratorios Bell de AT&T.

Año de aparición: 1977 - Actualidad

Descripción: Fue el intérprete usado en las primeras versiones de Unix y se convirtió en un estándar de facto.

Korn Shell (ksh)

Creador original: David Korn de los Laboratorios Bell de AT&T.

Año de aparición: Comienzos de los 80s - Actualidad.

Descripción: Escrito para el proyecto GNU (GPL). Sus funciones avanzadas para manejar archivos de comandos lo pusieron a la par con lenguajes de programación especializados como awk y perl.

Bourne-Again Shell (Bash)

Creador original: Brian Fox

Año de aparición: 1987 - Actualidad

Descripción: Escrito para el proyecto GNU, se usa por defecto en la mayoría de las distribuciones de Linux actualmente. La sintaxis de órdenes de Bash incluye ideas tomadas desde Korn Shell. (Tal y como hemos dicho, será Bash el que se usara en el curso).

Zsh

Creador original: Paul Falstad

Año de aparición: 1990 - Actualidad

Descripción: Se diseñó para poder usarse interactivamente. Reúne algunas importantes mejoras.

1.1 - Bash (Bourne Again Shell)

Bash es un intérprete escrito para el proyecto GNU, derivado del proyecto sh original (Bourne Shell) y será este intérprete en el que nos centremos en este curso casi exclusivamente.

Hacia 1977, el shell Bourne (sh) era distribuido exclusivamente en Unix versión 7 gracias a Stephen Bourne, que por entonces era investigador de los Laboratorios Bell y escribió el shell Bourne original. No fue hasta una década más tarde cuando Brian Fox escribió el shell Bash, en 1987. En 1990, Chet Ramey se convirtió en su principal desarrollador y mantenedor.

Actualmente, la programación de scripts para Bash resulta la forma más cómoda de crear scripts simples y automatizar tareas con ayuda de comandos del sistema, como cron. A lo largo de este curso veremos las formas más simples de introducirnos al mundillo del scripting en bash y su metodología de programación, bastante similar a cualquier otra que puedas conocer de antemano.

Para comenzar con la programación de scripts, deberemos tener en cuenta que es muy importante saber trabajar en la línea de comandos (*shell*), ya que cualquier trabajo que necesitemos realizar en ella, la podremos extrapolar a un script (tal y como haríamos, por ejemplo, en un fichero BATCH de Windows). La forma de ejecutar un script es lineal, pero nos podemos permitir el realizar saltos de una parte a otra del código por medio de llamadas a funciones y otros elementos que veremos a lo largo de este curso de iniciación, pero empezaremos por lo más sencillo para ir, progresivamente, adquiriendo soltura y ganando en dificultad.

Partamos de la base de que disponemos de una shell y dominamos ligeramente la introducción de comandos en ella, aunque sean comandos básicos como ls, ps, cd, pwd, mkdir, rm, etc... para poder navegar por el árbol de directorios de nuestra máquina y realizar tareas simples.

1.2 - Estructura básica de un script en Bash

Para iniciar la codificación de un script, lo primero que deberemos tener en cuenta es saber cómo definirlo para que el sistema sepa qué está leyendo cuando lo ejecutemos. Como ya sabemos, cualquier fichero en un sistema Linux es susceptible de ser ejecutado simplemente cambiándole el bit de ejecución de forma adecuada (aunque eso lo veremos un poco más adelante con el fin de aclarar el concepto).

Para identificar que el fichero es un script de Bash, la cabecera del mismo debe ir correctamente etiquetada. La forma adecuada para hacerlo es escribiendo una de las siguientes estructuras, indistintamente, al inicio del fichero:

#!/bin/bash

o

```
#!/bin/sh

# Aquí invocamos al enlace simbólico del intérprete por defecto de
nuestro usuario
```

Esta línea (con el uso de #! o *Shebang*) identifica nuestro fichero como un script del intérprete bash y realiza la llamada correspondiente al intérprete, normalmente ubicado en el directorio /bin/. Una vez identificado, podremos iniciar nuestro script a continuación, con la certeza de que el intérprete realizará correctamente su trabajo.

El uso de #!/bin/sh no está recomendado para entornos multiusuario donde no tengamos la certeza de que dicho enlace apunta al intérprete Bash, razón por la cual es mejor utilizar siempre la ruta absoluta de Bash que hemos visto anteriormente (#!/bin/bash).

1.3 - Nuestro primer script: “Hola Mundo”

Como en cualquier otro lenguaje de programación, la mejor forma de iniciar la andadura con el mismo es realizando y analizando un script sencillo con el ya famoso texto “Hola Mundo” (*Hello World* para los anglosajones).

Lo primero que haremos será abrir nuestro terminal o *shell* y crear un fichero (holamundo.sh) donde vayamos a escribir nuestro código inicial:

```
#!/bin/bash

# Invocamos nuestro primer comando de salida

echo “Hola Mundo”
```

Podemos usar cualquier editor de texto para ello, aunque se recomienda uno que resalte la sintaxis utilizada según el lenguaje de programación en el que estemos programando. Ahora, una vez guardados los cambios, deberemos modificar el permiso de ejecución del fichero recién creado con la orden:

```
chmod +x holamundo.sh
```

y ejecutarlo:

```
./holamundo.sh
```

La ejecución de este simple código nos devolverá por pantalla algo tan simple como el mensaje siguiente:

```
Hola Mundo
```

Vamos a explicar paso a paso lo que hemos realizado con este simple script:

- La primera línea, como ya hemos visto, solicita al sistema que sea el intérprete Bash el que ejecute el script.
- La segunda línea, al ir precedida por el símbolo almohadilla (#) indica que es un comentario. Cualquier línea precedida de un símbolo de almohadilla, será tomada como un comentario lineal y será ignorado en tiempo de ejecución del script.
- La tercera línea invoca al comando echo junto a una cadena de texto entrecomillada. Este comando escribe en la salida estándar la cadena de texto que se le pasa como parámetro, por lo que será de una tremenda utilidad como complemento a la hora de hacer *debugging* de nuestro script, como veremos más adelante.

1.4 - La línea de comandos: Introduciendo órdenes

El intérprete de comandos que incorporan la mayoría (por no decir el cien por cien) de distribuciones Linux y UNIX es tremendamente potente. Tanto, que nos permite realizar prácticamente cualquier tarea a través de comandos ejecutados en él. Aunque en todos los sistemas Linux disponemos de este genial elemento, como ya hemos visto anteriormente pero debemos saber acceder a él. La forma más habitual de reconocerlo es conociendo cuál es su estructura básica. Habitualmente nos encontraremos algo similar a lo siguiente (PROMPT):

[usuario@nombre_de_host ~]$

Aunque esta información puede ser fácilmente modificada en el fichero ./bashrc del usuario, lo normal es que mantenga esa estructura, indicando en primera instancia el nombre de usuario que está ejecutando el intérprete y a continuación el nombre de la máquina que alberga a dicho usuario, junto a la ruta donde nos encontramos dentro del sistema de archivos del equipo.

Los comandos más simples nos ayudarán a navegar por todo el sistema de archivos del disco o partición donde nos encontremos. Así pues, órdenes como “cd ..” o “ls” nos permitirán bajar un nivel o mostrar el contenido de un directorio concreto respectivamente si los invocamos en nuestro intérprete de comandos. La finalidad de un script es esencialmente la misma que la de dichos comandos, pero ejecutada una tras otra de la forma adecuada y secuencialmente con el fin de lograr un objetivo concreto. En nuestro script podremos hacer uso de cualquier comando que funcione en el Shell,

así como de variables que tomen el valor que necesitemos en cada momento, funciones que interpreten o ejecuten valores pasados por referencia, etc...

A bote pronto, esto puede resultar algo confuso, pero vamos a ver como aunar todos los conocimientos de programación para lograr nuestro objetivo: ahorrar tiempo y esfuerzo a la hora de realizar tareas repetitivas o complejas.

¡Importante!

Debemos tener claro el funcionamiento de un intérprete de comandos si queremos sacar el máximo provecho a nuestros scripts. De nada nos sirve saber la forma de realizar un script si no sabemos programarlo correctamente por no conocer la forma de invocar e introducir comandos en una shell e interpretarlos correctamente. Antes de ponerte a programar scripts en Bash, aprende a usar el intérprete para sacar todo su potencial.

1.5 - Conceptos sobre redirección de datos

La redirección de datos es un elemento indispensable a la hora de crear un script, ya que nos permite controlar, entre otras cosas, hacia donde se dirige la salida o desde donde obtenemos la entrada de datos de un script o una función concreta. El tener este tipo de control nos dará total libertad para manejar correctamente el flujo de datos de nuestro script. Vamos a ver unas breves pinceladas que nos permitirán adentrarnos en el concepto de redirección.

En Linux existen 3 descriptores de ficheros: stdin, stdout y stderr (std = estándar).

- **stdin** corresponde a la entrada estándar.
- **stdout** corresponde a la salida estándar.
- **stderr** corresponde al error estándar.

A través de estos descriptores, básicamente podremos realizar las siguientes tareas con el flujo de datos (i/o):

- Redirigir **stdout** a un fichero.
- Redirigir **stderr** a un fichero.
- Redirigir **stdout** a **stderr**.
- Redirigir **stderr** a **stdout**.
- Redirigir **stderr** y **stdout** a un fichero.
- Redirigir **stderr** y **stdout** a **stdout**.
- Redirigir **stderr** y **stdout** a **stderr**.

Cuando utilizamos el número 1, este *representa* a **stdout**, y el número 2, a **stderr**.

¡Atención!

Con el comando **less** podremos visualizar **stdout** (que permanecerá en el búfer) y **stderr**, que se imprimirá en la pantalla, pero será borrado si intentamos leer el búfer.

Ejemplo 1: Redirigir la salida estándar (**stdout**) a un fichero.

Si ejecutamos el siguiente comando en el terminal, hará que la salida de un programa se escriba en un fichero.

```
ls -l > ls-l.txt
```

En este caso, se creará un fichero llamado "**ls-l.txt**" que contendrá lo que se vería en la pantalla si escribiese el comando "**ls -l**" y lo ejecutase.

Ejemplo 2: Redirigir la el error estándar (**stderr**) a un fichero.

El siguiente comando hará que la salida **stderr** de un programa se escriba en un fichero.

```
grep da * 2> errores-de-grep.txt
```

En este caso, se creará un fichero llamado "**errores-de-grep.txt**" que contendrá la parte **stderr** de la salida (es decir, los errores) que daría el comando "**grep da ***".

Ejemplo 3: Redirigir la salida estándar (**stdout**) al error estándar (**stderr**).

Esto hará que la salida **stdout** de un programa se escriba en el mismo descriptor de fichero que **stderr**.

```
grep da * 1>&2
```

En este caso, la parte **stdout** del comando se envía a **stderr**.

Ejemplo 4: Redirigir el error estándar (**stderr**) a la salida estándar (**stdout**).

Este comando hará que la salida **stderr** de un programa se escriba en el mismo descriptor de fichero que **stdout**.

```
grep * 2>&1
```

En este caso, la parte **stderr** del comando se envía a **stdout**. Si hacemos una tubería con **less**, veremos que las líneas que normalmente "*desaparecen*" (al ser escritas en stderr), ahora permanecen (porque están en el stdout).

Ejemplo 5: Enviar el error estándar (**stderr**) y la salida estándar (**stdout**) a un fichero.

Este comando colocará toda la salida de un programa en un fichero. A veces, esto es muy conveniente utilizarlo en las entradas de **cron**, si necesitamos que un comando se ejecute en absoluto silencio.

```
rm -f $(find / -name temporal) &> /dev/null
```

Este comando (usado como una entrada de **cron**) eliminará todo archivo llamado "**temporal**" en cualquier directorio. Debemos estar muy seguros de lo que hace un comando si vamos a eliminar la salida por pantalla del mismo.

Todos estos ejemplos no son esenciales por si solos, pero si nos ayudarán a comprender el concepto de redireccionamiento del flujo de información dentro de un script. La forma más habitual de redireccionar en un script es por medio del uso de la salida estándar a un fichero, ya que nos ayuda a depurar (*debug*) y trazar errores en el código, así como a realizar registros (logs) del funcionamiento del mismo.

1.6 - Tuberías

Las tuberías (pipes) nos permiten enlazar la salida de un programa a la entrada del siguiente. Para comprenderlo claramente, lo mejor es ver un simple ejemplo:

```
ps aux | grep $USER
```

Este sencillo comando nos ofrece un listado de los procesos que se ejecutan en nuestra máquina, pero filtrado por nuestro nombre de usuario ($USER). Por lo que solamente aparecerán listadas todas aquellas líneas donde aparezca nuestro nombre de usuario del sistema.

Pero aún podemos hilar más fino... supongamos que necesitamos sacar un listado de todos los procesos filtrado por nuestro nombre de usuario en cuya instancia se esté ejecutando el navegador firefox. Una forma de hacerlo sería esta:

```
ps aux | grep $USER | grep firefox
```

Este comando solamente mostrará las instancias de firefox que esté ejecutando nuestro usuario (si hubiera más de uno logueado en el sistema, claro, ya que si no, no tiene mucha lógica).

Otro ejemplo sencillo:

```
ls -l | sed -e "s/[aeio]/u/g"
```

En este caso, ocurre lo siguiente: primero ejecutamos el comando **ls -l** y luego su salida, en lugar de imprimirse en la pantalla, se envía (entuba) al programa sed, que imprime la salida correspondiente a dicho comando. Concretamente, modifica las cuatro primeras vocales (a, e, i, o) por la “u”, dando un resultado bastante rocambolesco. ¡Haz la prueba!

2 - Uso de variables

Las variables son algo esencial en cualquier lenguaje de programación, ya que nos permiten almacenar cualquier dato para poder trabajar con él más adelante, guardándose durante la ejecución del script o programa, y llegando incluso a modificar el comportamiento del sistema si usamos las variables de ambiente del propio sistema en nuestro script.

2.1 - ¿Qué es una variable?

Una variable es un elemento que puede sufrir variaciones en su valor en tiempo de ejecución, a grandes rasgos. Esto quiere decir que podemos crear, modificar o eliminar una variable cuando lo necesitemos, como apoyo a la programación que estemos creando para conseguir un fin concreto.

En BashScript, las variables se definen sin necesidad de establecer el tipo de las mismas, ya que se hace de forma dinámica, es decir, no hace falta en principio indicar si son de tipo entero, cadena, booleano, etc... ya que es el propio sistema el que establecerá el tipo de valor de la variable en función de la necesidad de la operación. Para establecer el valor de una variable en BashScript, es tan sencillo como invocarla junto al valor que queremos darle. Las variables pueden ser creadas en cualquier momento, pero es mejor hacerlo justo antes de su primera utilización, escribiendo:

```
nombre_variable=valor_variable
```

Por ejemplo:

```
var1=1

var2=2
```

Aquí, hemos asignado el valor 1 a la variable **var1** y el valor 2 a la variable **var2**.

Si quisiéramos realizar algún tipo de operación con una variable (no solo mostrar su valor), ésta debe ir precedida del símbolo del dólar “$” que es lo que nos permite obtener el valor de la variable en el momento justo en el que lo necesitemos para usarlo adecuadamente.

```
let var3=$var1+$var2

echo “La variable var3 tiene un valor de: “ $var3
```

En este ejemplo, hemos establecido el valor de **var3** simplemente sumando (con el evaluador aritmético **let**) los valores de **var1** y **var2** y lo hemos mostrado con un **echo**, dando como resultado:

```
La variable var3 tiene un valor de: 3
```

Si creamos en una línea del script una variable con el valor de un comando y queremos imprimir la variable a la pantalla, podemos hacerlo así:

```
variable=`comando`

echo "$variable"
```

La palabra **$variable** está puesta entre comillas dobles para que se imprima todo el texto ignorando los espacios entre palabras.

Un ejemplo simple del uso de variables sería el ya conocido “Hola Mundo”:

```
#!/bin/bash

var=”Hola Mundo”

echo $var
```

La ejecución de este script nos daría como resultado el ya famoso “**Hola Mundo**”.

Con este sencillo concepto, podemos ir viendo de forma simple una aproximación al uso de variables en nuestro script. Veamos un ejemplo completo y su correspondiente salida:

```
#!/bin/bash

# Asignamos valores a las variables

var1=1

var2=2

# Mostramos los valores de las variables
```

```
echo "La variable var1 vale: " $var1

echo "La variable var2 vale: " $var2

# Realizamos la operación y la almacenamos en una nueva variable

let var3=$var1+$var2

# Mostramos el valor de la variable resultado

echo "La variable var3 tiene un valor de: " $var3
```

Salida obtenida en la ejecución del script:

```
La variable var1 vale: 1

La variable var2 vale: 2

La variable var3 tiene un valor de: 3
```

Debemos tener en cuenta que dos tipos diferentes de variables no podrán interactuar entre si. Por ejemplo, si queremos sumar el valor de la variable 1 (**$var1**) a la palabra "dos", que es una cadena de texto, el sistema nos dirá que es imposible y nos arrojará un error. Muchos de los errores mas comunes vienen de no tener un control del tipo de variable con el que estamos trabajando en cada momento, por lo que deberemos tener cuidado a la hora de realizar operaciones entre ellas y aprender a usar variables globales y locales. Las variables *locales* pueden crearse utilizando la palabra clave **local**. Veamos un ejemplo con nuestro ya conocido "Hola Mundo", modificandolo un poco para comprender mejor el concepto.

```
#!/bin/bash

HOLA="Hola"

function hola {

        local HOLA="Mundo"

        echo $HOLA

}

echo $HOLA            # Imprime el valor de HOLA global

hola                  # Imprime el valor de HOLA local (dentro de la
función)

echo $HOLA            # Imprime el valor de HOLA global otra vez
```

Con la ejecución de este ejemplo nos debería bastar para comprender el uso de una variable local. Vamos a analizarlo. Vemos como la salida del script es un tanto diferente a lo que cabría esperar:

```
Hola

Mundo

Hola
```

Seguro que esperabas un "**Hola**" y dos "**Mundo**", al creer que al invocar a la función, se ha modificado el valor de la variable global **HOLA** en tiempo de ejecución, ¿verdad?. Pues no es así. Esto es debido a que primero, se muestra el valor de la variable **HOLA** y luego se realiza una llamada a la función **hola()**, donde el valor de la variable **HOLA** es *local* (dentro de dicha función) y por tanto, no se sobre escribe al valor de **HOLA** global. Este concepto que te puede parecer algo abstracto ahora mismo, quedará mucho más claro cuando veamos el uso de las funciones.

2.2 - Variables intrínsecas de Bash

Estas variables son elaboradas por defecto por el propio Bash y son:

- $0 → Nombre del guión
- $1....$n → Variables que almacenan los n argumentos (opciones) proporcionados al comando invocado.
- $# → Variable que contiene el total de los argumentos proporcionados (devuelve un número).
- $* → Conjunto de los argumentos.
- $? → Valor de ejecución del comando anterior, si es cero es que el comando anterior se ejecutó sin errores, de lo contrario hubo algún error.
- $$ → Identifica el proceso del flujo.
- $! → Identifica el último proceso arrancado en el trasfondo (background).

Estas variables suelen usarse habitualmente cuando se invoca un comando externo o función interna del script. Es la mejor forma de *referenciar* (pasar por referencia) un parámetro para trabajar con él a través del código que estemos generando.

2.3 - Caracteres especiales

Existe un grupo de caracteres especiales (también llamados "meta-caracteres") que tienen un significado propio para Bash. Algunos de ellos son:

\ Le indica a Bash que ignore el carácter especial que viene después. Se suele usar para introducir funcionalidades al código, como saltos de línea, etc...

" Cuando se encierra entre comillas dobles un texto o una variable si esta es una frase
" (cadena de palabras) Bash lo interpretará como una cadena única.
$ Identifica que lo que sigue al carácter es una variable.
' ' Las comillas simples se usan para desactivar todos los caracteres especiales encerrados dentro de ellas, así tenemos que si escribimos '$variable', Bash interpretará literalmente lo escrito y no su valor como variable.
Cuando se coloca este carácter dentro de una línea del guión, Bash ignora el resto de la línea. Muy útil para hacer comentarios y anotaciones o para inhabilitar una línea de comandos al hacer pruebas.
; Este carácter se usa para separar la ejecución de distintos comandos en una misma línea de comandos.
`` Se utiliza para convertir la salida de un comando en una variable. El comando en cuestión se ejecuta en una sub-shell.

Las comillas simples y el carácter “\“ hacen que Bash ignore la función de cualquier carácter especial, lo que se llama “*escapado*” o escape. Aunque la diferencia entre ambos es que las comillas simples harán que se ignore todos los meta-carácteres encerrados entre ellas y el carácter “\“solo el que sigue después.

También disponemos de un listado con algunas de las *opciones genéricas* para utilizar junto a escape con las que trabajar en Bash:

\a Carácter de campana ASCII (07)
\d Fecha en formato día
\e Carácter de escape ASCII (033)
\h Nombre de la máquina (solo host)
\H Nombre completo de la máquina (FQDN)
\n Carácter de nueva línea (salto de línea)
\r Retorno de carro
\s Nombre de la shell, el nombre base de $0
\t Hora actual en formato 24h.(HH:MM:SS)
\T Hora actual en formato 12h. (HH:MM:SS)
\@ Hora actual en formato 12h. (AM/PM)
\u Nombre del usuario actual ($USER)
\v Versión de Bash
\V Versión del paquete Bash (Versión + patch-level)
\w Directorio actual de trabajo (como pwd)
\W Nombre base del directorio actual de trabajo
\! Número del comando actual en el histórico de comandos (history)
Numero de comando del comando actual
\$ Si el UID efectivo es 0, tiene un valor #; en cualquier otro caso, vale $
\nnn Carácter correspondiente al número en octal *nnn*
\\ Contra-barra
\[Inicio de una secuencia de caracteres no imprimibles que pueden usarse para incrustar una secuencia de control del terminal en el PROMPT
\] Fin de una secuencia de caracteres no imprimibles

2.4 - Palabras especiales

Hay un grupo de palabras que tienen significado especial para Bash y que siempre que se pueda deberá evitarse el uso cuando se escriben líneas de comandos para no crear "confusiones". Algunas de las más importantes son:

exit	Se sale del flujo de ejecución
break	Se envía explícitamente a salir de un bucle
continue	Se envía explícitamente a retornar en un bucle
return	Como **exit** pero solo se sale del comando u operación sin cerrar el flujo
true	Indica que una condición es verdadera
false	Indica que una condición es falsa

2.5 - Asignando valores a variables por interacción del usuario

Muchas veces nos hará falta asignar el valor de una variable “al vuelo” durante la ejecución del script por medio de un usuario. Para ello, tendremos que usar la orden “**read**”, que nos permitirá paralizar el script momentáneamente y esperar a que el usuario haya introducido el valor requerido para almacenarlo en una variable concreta. Si hacemos un **read** sin asignar variable, el dato se almacenará en **$REPLY**, una variable del sistema. No es bueno abusar de esta orden en un script, puesto que perderemos parte de la automatización que buscamos al tener que parar la ejecución del script cada poco tiempo para solicitar nuevos datos al usuario.

El modo de uso de la orden read es el siguiente:

```
read variable
```

Donde “**variable**” será la variable donde se almacenará el valor que el usuario inserte. Por si solo, esto no significa nada para el usuario que está ejecutando el script, ya que no ofrece ningún aviso de que está esperando una interacción. Por ello, lo mejor es utilizarlo con la opción siguiente:

```
read -p “Introduce el valor requerido y pulsa Enter: ” var
```

Esto muestra el mensaje “*Introduce el valor requerido y pulsa Enter*” para que el usuario realice la acción correspondiente y así pueda almacenar el valor introducido por el usuario en la variable “*var*”. El valor de la variable solo se almacenará cuando el usuario presione la tecla “Enter”. A continuación podemos validar visualmente el valor de la variable para que el usuario compruebe que efectivamente, el valor es el correcto, usando la orden **echo**:

```
echo “Has establecido un valor de “ $var “ para la variable var”.
```

Lo que nos daría una salida como la siguiente al ejecutar el script:

```
Introduce el valor requerido y pulsa Enter: 42

Has establecido un valor de 42 para la variable var.
```

Si quisiéramos realizar alguna comprobación adicional, necesitaríamos otra clase de estructura para controlar el valor, como una estructura de control, la cual veremos un poco más adelante.

Es posible que necesitemos que el texto a introducir por parte del usuario no se muestre (por ejemplo, si queremos que escriba un PIN (*Personal Identification Number*) y no queremos que se muestre por pantalla). En ese lugar, necesitaremos una orden adicional al invocar a read:

```
read -s -p “Introduce el PIN: “ pin1
```

Esto nos permitirá almacenar en una variable el valor de una contraseña (PIN) introducida por el usuario, pero de forma que no aparezca mientras se escribe. Para cerciorarnos que el usuario ha escrito correctamente el PIN, podemos almacenar una segunda vez el PIN en otra variable para luego compararlos con una estructura de control y si son iguales, continuar con el script o, en caso contrario, volver a solicitar el número al usuario.

2.6 - Operadores

Antes de meternos de lleno con las estructuras de control, conviene que sepamos cómo usar los operadores a la hora de realizar las diferentes comprobaciones en nuestro código. Es muy importante conocerlos todos para aprovecharlos al máximo. Disponemos de diferentes operadores con los que trabajar y los hemos dividido en varias tablas para su mejor comprensión y localización.

Para realizar comparaciones en BashScript, únicamente deberemos tener en cuenta los siguientes operadores:

Comparación de enteros (*números*)

-eq	Equal (es igual a)	if ["$a" -eq "$b"]
-ne	Not Equal (no es igual a / es distinto a)	if ["$a" -ne "$b"]
-gt	Greater Than (es mayor que)	if ["$a" -gt "$b"]
-ge	Greater Equal (es mayor que o igual a)	if ["$a" -ge "$b"]
-lt	Lower Than (es menor que)	if ["$a" -lt "$b"]
-le	Lower Equal (es menor que o igual a)	if ["$a" -le "$b"]
<	Menor que (usar con doble paréntesis)	(("$a" < "$b"))
<=	Menor que o igual a (usar con doble paréntesis)	(("$a" <= "$b"))
>	Mayor que (usar con doble paréntesis)	(("$a" > "$b"))
>=	Mayor que o igual a (usar con doble paréntesis)	(("$a" >= "$b"))

Comparación de cadenas

=	Igual a	if ["$a" = "$b"]
==	Igual a	if ["$a" == "$b"]
!=	No igual a / Distinto a	if ["$a" != "$b"]
<	Menor que (orden alfabético ASCII)	if [["$a" < "$b"]]
>	Mayor que (orden alfabético ASCII)	if [["$a" > "$b"]]
-z	Cadena vacía (null) o de longitud cero	if [-z “$cadena”]
-n	Cadena no vacía (con contenido)	if [-n “$cadena”]

Apunte 1: Aunque “=” es un sinónimo del operador “==”, se comporta diferente cuando se usa dentro de corchetes dobles en lugar de simples, por ejemplo:

```
[[ $a == b* ]]   # Verdadero si $a empieza con una "b" (expresión
regular coincide).

[[ $a == "b*" ]] # Verdadero si $a es igual a b* (coincidencia
literal).


[ $a == b* ]     # Verdadero si existe división de palabras.
```

```
[ "$a" == "b*" ] # Verdadero si $a es igual a b* (coincidencia
literal).
```

Apunte 2: Los operadores “<” y “>” necesitan ser escapados dentro de corchetes simples.

```
if [ "$a" \< "$b" ]

if [ "$a" \> "$b" ]
```

Todas las condiciones pueden negarse (**! condición**), sumarse (*verdadero* si se cumple alguna de las condiciones, p.ej. **condicion1 || condicion2**) o multiplicarse (*solo será verdadero* si ambas condiciones son ciertas, p. ej. **condicion1 && condicion2**). Son las llamadas “*Comparaciones lógicas*”. Veamos los tipos existentes:

-a	Y lógico (and) devuelve “true” si ambas son verdaderas.	exp1 -a exp2
-o	O lógico (or) devuelve “true” si alguna de las expresiones son verdaderas.	exp1 -o exp2

Éstos últimos operadores son similares respectivamente a los operadores de Bash “**&&**“ (and) y “||” (or) cuando se usan con doble corchete:

```
[[ condicion1 && condicion2 ]]

[[ condicion1 || condicion2 ]]
```

Operadores de ficheros

-e	Verdadero si el fichero existe	if [-e “$fichero”]
-L	Verdadero si existe y es un enlace simbólico	if [-L “$fichero”]
-d	Verdadero si existe y es un directorio	if [-d “$directorio”]
-f	Verdadero si existe y es un fichero regular	if [-f “$fichero”]
-r	Verdadero si existe y se puede leer (read)	if [-r “$fichero”]
-w	Verdadero si existe y se puede	if [-w

	escribir (write)	“$fichero”]
-x	Verdadero si fichero existe y se ejecutable	if [-x “$fichero”]
-O	Verdadero si $USER es dueño del fichero	if [-O “$fichero”]
-G	Verdadero si el grupo es igual al de $USER	if [-G “$fichero”]

Estas operaciones con ficheros son las más comunes cuando trabajamos con un script, aunque podemos usar algunas más que nos resultarán útiles en alguna que otra ocasión:

fichero1 -nt fichero2 → Verdadero si fichero1 más actual (por fecha de modificación) que fichero2.

fichero1 -ot fichero2 → Verdadero si fichero1 más antiguo que fichero2.

fichero1 -ef fichero2 → Verdadero si fichero1 y fichero2 tiene el mismo número de *device* e *inode*.

2.7 - Colores

Como en cualquier otro lenguaje, en BashScript también podemos utilizar coloreado en el texto con el fin de resaltarlo y usarlo para nuestros propósitos. Con tal fin, disponemos de la siguiente codificación:

Negro 0;30	Gris oscuro 1;30
Azul 0;34	Azul claro 1;34
Verde 0;32	Verde claro 1;32
Cían 0;36	Cían claro 1;36
Rojo 0;31	Rojo claro 1;31
Purpura 0;35	Purpura claro 1;35
Marrón 0;33	Amarillo 1;33
Gris claro 0;37	Blanco 1;37

Utilizarlo es bastante simple, pero requiere de algo de paciencia a la hora de colorear texto. El comando echo se debe de ejecutar con el parámetro -e haciendo que Bash interprete el comando \e (o sea el escape) de la siguiente manera:

```
"\e[CODIGOmtexto\e[0m"
```

Veamos un ejemplo sencillo:

```
echo -e "Esto es \e[1;31mrojo claro\e[0m y esto es \e[1;34mazul
claro\e[0m"
```

Nos dará un resultado exactamente igual a este:

```
Esto es rojo claro y esto es azul claro
```

En resumen para poder utilizarlo solo hay que poner los caracteres o cadenas que quieras que tengan color entre los caracteres de escape (escaparlos).

Como curiosidad, copia y ejecuta el siguiente código en un fichero de script para comprobar lo expuesto:

```
#!/bin/bash

echo -e "Así se escribe
\e[1;34mG\e[0m\e[1;31mo\e[0m\e[1;33mo\e[0m\e[1;34mg\e[0m\e[1;32ml\e[0m
\e[1;31me\e[0m"
```

2.8 - Evaluadores aritméticos

Cuando en un script necesitamos realizar cálculos aritméticos simples (es decir, que una expresión sea evaluada de forma aritmética) disponemos de dos opciones principales, **let** y **expr**.

La orden **let** nos permite realizar cálculos simples entre valores de variables o números, por ejemplo:

```
let var3=$var1+$var2
```

Con **expr** podemos realizar también una serie de operaciones en tiempo de ejecución de forma sencilla, por ejemplo:

```
echo "Sumar 2 y 2 da lugar a " $(expr 2 + 2)
```

Los espacios son importantes al utilizar **expr**. Este ejemplo da un resultado como la siguiente salida:

```
Sumar 2 y 2 da lugar a  4
```

Pero si no dejamos espacios al utilizar el operando:

```
echo "Sumar 2 y 2 da lugar a " $(expr 2+2)
```

Dará un resultado erróneo:

```
Sumar 2 y 2 da lugar a  2+2
```

Esta salida no será algo que podamos utilizar. Así, según lo que valgan las variables a operar, podremos utilizar los operandos simples que se reflejan en la siguiente tabla:

var++ y var--	Se procesa el valor y después se aumenta o decrementa en uno el valor
++var y --var	Aumenta o decrementar el valor de la variable en uno y se procesa
- +	Menos y más (unario)
! ~	Negación lógica y de bits
******	Exponencial
*** / %**	Multiplicar, dividir y resto
+ -	Sumar y restar
<< >>	Rotar bit a izquierda y derecha
<= >= < >	Comparaciones
== !=	Igualdad, desigualdad
&	Operación a nivel de bit AND
^	Operación a nivel de bit exclusivo OR
|	Operación a nivel de bit OR
&&	AND lógico
||	OR lógico
`expr1?expr2:expr3`	Evaluación condicional: Si expr1 entonces expr2, si no expr3

En la Wikipedia podemos ampliar información acerca de las operaciones lógicas a nivel de bit.

¡Atención!

Algunos caracteres de la tabla deben ser escapados para evitar que Bash no los interprete al mostrarlos. Por ejemplo, si queremos simplemente mostrar < y > tendremos que escaparlos así: \< y \> . De no ser así, puede darse el caso que intente redirigir la salida estándar a un fichero u obtener la entrada estándar como método de introducción de información en el script, ya que estos caracteres en concreto se utilizan para eso.

Con estos conocimientos básicos para comenzar a programar scripts, ya podremos afrontar cualquier código inicial de BashScript. En el siguiente tema haremos uso de las estructuras de control en nuestros scripts para dotarlos de potencia y dinamismo. Vamos a ello...

3 - Estructuras de control

Hemos mencionado las estructuras de control, pero vamos a ver qué son y cómo pueden ayudarnos. Las estructuras de control son elementos de programación que pueden controlar y modificar el flujo de ejecución de las instrucciones de un programa. Como definición, puede resultarnos un tanto vaga, pero en esta unidad vamos a ver cómo funcionan con algunos ejemplos prácticos y de qué forma podemos usarlas para controlar el flujo de ejecución en nuestros scripts y obtener lo que necesitamos.

Según la Wikipedia, con las estructuras de control podremos:

- de acuerdo a una condición, ejecutar un grupo u otro de sentencias (If-Then-Else, Select y Case)
- Ejecutar un grupo de sentencias mientras exista una condición (While-do)
- Ejecutar un grupo de sentencias hasta que exista una condición (Until-do)
- Ejecutar un grupo de sentencias un número determinado de veces (For-Next)

Todas las estructuras de control tienen un único punto de entrada y un único punto de salida durante la ejecución del código. Las estructuras de control se puede clasificar en:

1. Secuenciales.
2. Iterativas.
3. De control avanzadas.

Esto es una de las cosas que permite que la programación se rija por los principios de la programación estructurada.

Los lenguajes de programación modernos tienen estructuras de control similares. Básicamente lo que varía entre las estructuras de control de los diferentes lenguajes es su sintaxis, cada lenguaje tiene una sintaxis propia para expresar la estructura.

3.1 - if-then-else-fi

Si necesitamos realizar una acción u otra en base a una condición dada, deberemos utilizar la estructura *if-then-else-fi*. En la mayor parte de lenguajes de programación, las estructuras de control condicionales se invocan de forma muy similar, aunque sintácticamente difieren un poco entre los diferentes lenguajes. Veámoslo con un ejemplo conceptual...

En algunos lenguajes de programación, la estructura de control if la podríamos utilizar así:

```
IF (Condición) THEN
      (Si se cumple la condición principal, ejecuta sentencia 1)
ELSE
      (Si no, ejecuta sentencia 2)
END IF
```

En BashScript, la forma correcta de finalizar la estructura es ligeramente diferente:

```
  IF (Condición) THEN
       (Si se cumple la condición, ejecuta sentencia 1)
  ELSE
       (Si no, ejecuta sentencia 2)
  FI
```

Como podemos observar, la estructura es idéntica, pero cambia alguna cosa (se usa **IF**, **ELSE**, **FI** en lugar de **IF**, **ELSE**, **END IF**), aunque el resultado debería ser el mismo en igualdad de condiciones.

Vamos a ver la manera más sencilla de usar una estructura de control:

```
if [ 1 -gt 2 ]; then
       echo "Este caso es imposible puesto que 1 nunca será mayor que
2."
else
       echo "2 siempre es mayor que uno."
fi
```

En este ejemplo, la primera condición jamás se podrá dar, ya que 1 jamás podrá ser mayor (-gt) que 2, mientras que la segunda condición será la que siempre se muestre.

Otro caso un poco más esclarecedor...

```
#!/bin/bash

read -p "Introduce un número del 1 al 9: " num

if [ “$num” -gt “5” ]; then

       echo "El número introducido es MAYOR que 5."

else

       echo "El número introducido es MENOR que 5."

fi
```

Aquí estamos realizando una comparación entre un número a elección del usuario y el número 5, de tal forma que si el número es mayor que 5, se cumple la primera condición, y si no lo es, se cumplirá la segunda condición, mostrando los respectivos mensajes.

3.1.1 - Ejemplos con if-else-fi

Veamos un caso práctico utilizando el ejemplo del final del apartado 2.5, el ejemplo de la comprobación por comparación de números PIN almacenados en variables:

```
#!/bin/bash

# Leemos los números PIN y los almacenamos en variables

read -s -p "Introduce el PIN: " pin1

# Escapamos el salto de línea ya que la opción -s de read los limita

echo -e "\nVariable 1 Almacenada."

read -s -p "Vuelve a introducir el PIN: " pin2

# Escapamos de nuevo el salto de línea

echo -e "\nVariable 2 Almacenada."

# Comparamos el valor de ambas variables y mostramos el resultado

if [ $pin1 -eq $pin2 ] ; then
       echo "El PIN coincide."
else
       echo "El PIN no coincide."
fi
```

Cuando lo ejecutemos, si las variables numéricas son las mismas, obtendremos una salida exactamente igual a esta:

```
Introduce el PIN:
Variable 1 Almacenada.
Vuelve a introducir el PIN:
Variable 2 Almacenada.
El PIN coincide.
```

Si durante la ejecución de esta parte del script, el usuario introduce otra cosa que no sea un número, el script dará un error, ya que las comparaciones con **-eq** solo permite hacerlas entre números y no entre cadenas de caracteres. Si quisiéramos usar una cadena de caracteres en lugar de números, la comparación debería ser con el operador “=” o “==”.

```
#!/bin/bash

# Comprobación de contraseñas con estructuras de control

read -s -p "Introduce contraseña: " pass1 ;

echo -e "\nVariable 1 Almacenada."

read -s -p "Vuelve a introducir contraseña: " pass2

echo -e "\nVariable 2 Almacenada."

# Comparamos los valores de las variables entre si y mostramos el
resultado

if [ "$pass1" == "$pass2" ] ; then
       echo "La contraseña coincide."
else
       echo "La contraseña no coincide."
fi
```

¡Atención!

Observa que las estructuras de control están correctamente "identadas" (enlace a Wikipedia), es decir, están estructuradas a base de espacios o tabulaciones (sangrado) para poder delimitar mejor las sentencias internas de cada estructura y no equivocarse a la hora de realizar cambios. Esto lógicamente surte un mayor efecto cuando existen muchas estructuras de control *anidadas* (es decir, una dentro de otra), con el fin de delimitar donde acaba y empieza cada una. No te preocupes si no entiendes el concepto, ya que a medida que avances, irás asimilándolo de forma natural y no podrás vivir sin la "identación" de código. De hecho, los programas realizados en lenguajes de programación como *Python* no funcionan si no están correctamente *identados*.

Veamos otro ejemplo algo más complejo para arrojar algo de luz. Como puedes observar, vamos poco a poco, asimilando conceptos. En este caso vamos "*anidar*" elementos de control en nuestro código para realizar diferentes comprobaciones; En el siguiente ejemplo, anidaremos dos estructuras de control. La primera nos dirá si la cadena que introduce el usuario está vacía, en cuyo caso, saldrá del script. La segunda comprobación en caso de que la cadena tenga contenido (no sea *null*), es para identificar si el número es menor de 10. En caso afirmativo, se mostrará el número elegido por el usuario.

Ejercicio, realizar la modificación en este script, llamado control3.sh, para controlar que el campo nombre o el campo apellidos no contenga vacíos, en caso de que uno de los dos campos contenga un valor vacio, finalizaremos nuestro script.

```
#!/bin/bash

#Limpiamos la pantalla.

clear

#Capturamos el valor de una variable

echo "Introduce tu nombre"

read nombre

echo "Introduce tu apellido"

read apellido

echo "Hola "$nombre $apellido"."

echo "Elige un número del 1 al 10"

read num

if [ -z $num ] ; then
        echo "No has introducido ningún número."
        echo "Saliendo..."
        exit 0
else
        if  [[ $num -gt "10" || $num -lt “1” ]] ; then
                echo "El número elegido ("$num") no es válido."
                echo "El rango es entre 1 y 10."
        else
                echo "El número que has elegido es el: "$num
        fi
fi
```

Otra vez volvemos a remarcar la importancia del *identado* de código. La estructura coloreada en azul es la principal, y es en su interior (en el *if anidado*) desde donde realizamos la segunda llamada a la estructura de control coloreada en rojo. Lo que hacemos es realizar “dos preguntas”. La primera para ver si la variable tiene contenido, donde, en caso afirmativo, realizaremos la segunda comprobación acerca de si el número es mayor de 10 o menor que 1. Por eso es tremendamente importante alinear las estructuras en la vertical correspondiente, ya que si todo estuviera alineado a la izquierda, nos sería muy difícil saber a qué estructura pertenece cada elemento cuantos más elementos tuviera un script.

3.1.2 - Otras formas de utilizar la sentencia if

Además de los corchetes, a la hora de escribir código también podemos utilizar las sentencias de control de la siguiente forma:

```
cadena1="Texto"

cadena2="Otro texto"

if test "$cadena1" = "$cadena1"

then

       echo "Cadenas iguales"

else

       echo "Cadenas distintas"

fi
```

Este método, igualmente válido a la hora de realizar el control del flujo de ejecución, nos arrojaría el mismo resultado que el siguiente código:

```
cadena1="Texto"

cadena2="Otro texto"

if [ "$cadena1" == "$cadena1" ]

then

       echo "Cadenas iguales"

else

       echo "Cadenas distintas"

fi
```

En realidad, los corchetes son un atajo al programa test, que es el que ejecuta la acción de comparación.

También podemos usar ((condición)) para provocar una evaluación aritmética de una condición dada. Esto mismo podremos lograrlo igualmente con **let** y **expr** como ya hemos visto anteriormente. Para operaciones matemáticas más complejas, lo ideal es hacer uso de **bc**, aunque este comando merece un curso aparte y no lo veremos aquí.

3.1.3 - Usando elif en una estructura if

El propósito de crear un script es el de *realizar una operación repetitiva* (automatizar) que requeriría mucho tiempo hacerla manualmente, aunque también queremos que nuestro código sea limpio y ocupe lo menos posible. Cuando más limpio y optimizado esté nuestro código, menos gasto de procesador y memoria consumirá durante su ejecución, lo que dejará más recursos libres al resto de procesos activos en el sistema que puedan ser críticos o simplemente necesitar mayor consumo de CPU. Por ello, aprender a optimizar nuestro código y a mantenerlo limpio y ordenado (comentándolo en caso necesario) nos ayudará a mejorar a la hora de crear mejores y más potentes scripts. Una forma de evitar ensuciar código innecesariamente es utilizando **elif**, cuya estructura básica sería algo así:

```
IF "condicion" THEN

       "comandos"

ELIF "condicion" THEN

       "comandos"

ELSE

  "comandos"

FI
```

Esta estructura podemos verla en acción en el siguiente ejemplo para aclarar un poco el concepto:

```
#!/bin/bash

hora1=`date +%k`  # Obtenemos la hora en formato 24H

hora2=`date +%l`  # Obtenemos la hora en formato 12H

if [ $hora1 -eq 12 ] ; then

       dia="del mediodía."

elif [ $hora1 -gt 12 ] ; then

       dia="de la tarde."

else

       dia="de la mañana."

fi

echo "Son las $hora2 $dia"
```

Si nuestro sistema tiene una salida del comando date, por ejemplo, similar a esta:

```
mar jun 18 16:50:55 CEST 2013
```

La salida del script anterior será:

```
Son las 4 de la tarde.
```

A grandes rasgos, lo que hacemos con **elif** es contar con la salida anterior y utilizarla para ahorrarnos un condicional. Si lo leemos de forma interpretada (en lenguaje coloquial, podríamos decir), tendríamos la siguiente orden:

Si la hora del día (en formato 24 horas) es igual a 12 (es decir, la condición evaluada es verdadera), la variable "dia" corresponde a "del mediodía". Si no es igual a 12, pero es mayor de 12, le corresponde el valor "de la tarde" y si no es ninguna de las dos anteriores condiciones, entonces obtiene el valor "de la mañana" por eliminación. El resultado nos muestra la hora en formato 12h junto al valor de la variable según lo calculado.

Como vemos, no ha hecho falta anidar ninguna estructura dentro de la principal, por lo que ganaremos algo en limpieza. Esto en una simple estructura aislada no es mucho, pero en scripts de varias páginas nos supondrá unas cuantas líneas de ahorro y menos quebraderos de cabeza con la identación. Cabe añadir que podemos utilizar varios **elif** en la misma estructura de control según necesitemos para establecer condiciones adicionales a evaluar:

```
IF "condicion" THEN

       "comandos"

ELIF "condicion" THEN

       "comandos"

ELIF "condicion" THEN

       "comandos"

ELIF "condicion" THEN

       "comandos"

ELSE

  "comandos"

FI
```

...aunque usar muchos **elif** no es recomendable según el caso, puesto que existen otros métodos mejores de conseguir los mismos resultados con menos código. La experiencia te dirá qué usar en cada caso.

3.2 - Condicionales por interacción del usuario (Select - Case)

Existen ciertos tipos de condicionales diferentes a los **if** que requieren de interacción por parte del usuario la mayoría de las veces para, en base a una elección o selección programada, ejecutar una u otra cosa. Vamos a verlos...

3.2.1 - Select

Esta estructura está a medio camino entre una orden de lectura de entrada de datos y una de control de flujo. Se utiliza principalmente para que el usuario elija una opción entra una lista prefijada con anterioridad, volviendo al inicio hasta que metamos un break para salir. Su sintaxis es muy sencilla:

```
SELECT variable IN elemento1 elemento2 elemento3 elemento4

DO

        (Se ejecuta el comando)

DONE
```

Básicamente, esta orden SELECT nos mostrara por la salida estándar una lista numerada (de 1 a n elementos, es decir, tantas opciones como elementos le pasemos) que contendrá todas las palabras después de in (en este caso: elemento2 elemento3 elemento4). Luego se quedara esperando nuestra elección. Si tecleamos uno de los números correspondientes a alguna de las palabras en la lista, se le asignara dicho valor a la variable y se ejecutaran los comandos. Si tecleamos un número fuera del rango o cualquier otra cosa, la variable se quedara sin contenido. Lo tecleado se almacenara en la variable $REPLY. Los comandos se ejecutaran cada vez que elijamos una opción hasta que ordenemos un comando break que interrumpa la ejecución de la orden.

Si no indicamos la lista de palabras, se nos mostrara una lista con los parámetros pasados al comando por medio de las variables de referencia $1-$9.

Vamos a ver un ejemplo junto a la herramienta CASE (que es la forma habitual de utilizarlo para realizar selecciones, aunque no la única) y así iremos captando el concepto global de estas dos herramientas:

```
#!/bin/bash

echo "Ejemplo de estructuras SELECT y CASE"

echo "Selecciona una acción:"

select ACCION in Empezar Repetir Acabar

do

        case $ACCION in
```

```
        "Empezar")
              echo "El usuario quiere empezar."
        ;;
        "Repetir")
              echo "El usuario quiere repetir."
        ;;
        "Acabar")
              echo "El usuario quiere salir."
              break
        ;;
        *)
              echo "No sabemos lo que quiere el usuario."
        ;;
        esac
done
```

La *identación* del código en este caso nos ayuda bastante a identificar donde empieza y acaba la estructura SELECT y donde estamos utilizando la estructura CASE. La salida de este script una vez ejecutado sería la siguiente:

```
Ejemplo de estructuras SELECT y CASE
Selecciona una acción:
1) Empezar
2) Repetir
3) Acabar
#? 1
El usuario quiere empezar.
#? 2
El usuario quiere repetir.
#? 3
El usuario quiere salir.
```

Como podemos observar, la ejecución del script no se para hasta que encuentra la orden break en el patrón establecido para salir. Esto nos permite seguir ejecutando indefinidamente el script mientras sea útil. Hay que tener en cuenta que si se nos olvida incluir una salida en alguna o varias opciones, se nos generaría un bucle de ejecución del que solamente podríamos salir parando el proceso del mismo (con un Ctrl+C, por ejemplo).

3.2.2 - Case

Hemos visto de forma breve cómo funciona la estructura **SELECT** y sabemos que una buena forma de utilizarla es junto a **CASE**... pero aún no sabemos para qué se utiliza esta última estructura, aunque podemos intuirlo. Despejemos las dudas. Con Case podemos comparar el valor de una variable con otros valores distintos y actuar en consecuencia. Su sintaxis es la siguiente:

```
CASE $variable IN
        patron1 )
                (Se ejecuta el comando 1)
        ;;
        patron2 )
                (Se ejecuta el comando 2)
        ;;
        patron3 )
                (Se ejecuta el comando 3)
        ;;
 ESAC
```

Este código de corte genérico nos permite realizar una selección simple entre *n* elementos, los cuales debemos enumerarlos por separado junto a la acción que tendrá que desarrollarse si se selecciona dicho patrón. Para muestra, un botón:

```
#!/bin/bash
echo "  1. Sumar 5+5"
echo "  2. Restar 8-4"
echo "  3. Multiplicar 10x10"
echo "  4. Dividir 4/2"
```

```
read -p "Realiza tu selección: " selec

case $selec in

       1)

             echo "El resultado es 10"

       ;;

       2)

             echo "El resultado es 4"

       ;;

       3)

             echo "El resultado es 100"

       ;;

       4)

             echo "El resultado es 2"

       ;;

esac
```

En este sencillo ejemplo, según lo que elija el usuario y como el resultado de las operaciones ya lo hemos preestablecido de antemano, nos aparecerá directamente al seleccionar la opción deseada. Al no incorporar ningún método de control del flujo de información, cuando se seleccione la opción y se ejecute el comando para mostrar la información solicitada, saldremos del script. Por ello es bueno utilizarlo con métodos de control del flujo de ejecución (o la propia sentencia **SELECT** como ya hemos visto en un ejemplo anterior) si queremos mantenernos dentro del script hasta seleccionar una opción concreta.

3.3 - While

Este condicional de repetición (bucle) es tremendamente útil para recorrer una lista mientras la evaluación de la operación sea verdadera, es decir, con while repetiremos una lista de comandos siempre que la condición sea cierta. En el momento en que la condición evaluada no se adecúe a la condición inicial, saldremos del bucle. Hay que recordar que con while se comprueba la condición antes de ejecutar los comandos, con lo cual puede que no se ejecuten ni una sola vez. También disponemos de la estructura until, cuyo funcionamiento es similar a while, solo que en esta ocasión la condición sera negada (es decir, se repetirá mientras la condición evaluada sea falsa, en vez de repetirla mientras la condición sea cierta,como sucedería con while).

La estructura while tiene una sintaxis muy simple:

```
WHILE [ condición ];
```

```
DO
  (comando1)
  (comando2)
  (comando3)
   ....
DONE
```

La forma coloquial de expresarla sería "*mientras que la condición sea la indicada, ejecuta los siguientes comandos*". Como podemos observar, podemos introducir tantos comandos a ejecutar como necesitemos, pero en el momento en que se modifique la condición inicial de la línea principal de while, se interrumpirá el flujo de información dentro del bucle y volverá a la ejecución principal del script.

El ejemplo más básico que podemos escribir para comprender su funcionamiento podría ser este:

```
#!/bin/bash
num=0
while [ $num -le 10 ]; do
       echo "\$num: $num"
       let num=$num+1
done
```

Cuya salida al ejecutarlo arrojaría el siguiente resultado:

```
$num: 0
$num: 1
$num: 2
$num: 3
$num: 4
$num: 5
$num: 6
$num: 7
$num: 8
$num: 9
$num: 10
```

A fin de cuentas, lo que estamos pidiéndole a while es que ejecute el comando mientras el valor de la variable sea menor de diez (10). En cada ejecución del bucle (repetición)

le vamos sumando 1 al valor de la variable num. Eso hace que aumente secuencialmente hasta llegar al valor de ruptura del bucle.

Otra forma de utilizar while es por medio de variables de control externas, es decir, usamos el valor de una variable para controlar el flujo de ejecución *DENTRO* del bucle. Por ejemplo, necesitamos que el bucle se ejecute mientras el valor de una variable sea 1. Por medio de un condicional, le podemos decir al bucle que deje de ejecutarse simplemente cambiando el valor de la variable de control al darse la condición precisa.

```
#!/bin/bash

ctrl=0   # Inicializamos la variable de control

while [[ $ctrl = "0" ]]; do

       read -p "Introduce el número 5: " num

       if [[ "$num" -eq "5" ]]; then

             echo "El número introducido es el 5!!"

             ctrl="1"  # Interrumpimos del bucle cambiando el valor de
la variable

       else

             echo "El número introducido no es el 5"

       fi

done
```

Su salida es esclarecedora, ya que vamos viendo el flujo de información a través del código:

```
Introduce el número 5: 2

El número introducido no es el 5

Introduce el número 5: 8

El número introducido no es el 5

Introduce el número 5: 4

El número introducido no es el 5

Introduce el número 5: 32

El número introducido no es el 5

Introduce el número 5: 5
```

```
El número introducido es el 5!!
```

También podemos hacer que el usuario nos facilite el valor de la variable que condicionará la ejecución del bucle. Veamos un ejemplo:

```
#!/bin/bash

read -p "Elige un número entre 1 y 50: " num

if [[ $num -lt 1 || $num -gt 50 ]]; then

       echo "El número está fuera de rango."

else

       while [[ $var != $num ]]; do

                     let var=$var+1

                     echo "El número elegido NO es "$var

             done

             echo "El número elegido es " $var

fi
```

En el ejemplo, necesitamos que el usuario introduzca un número entre el 1 y el 50, el cual usaremos como variable de control de flujo. Primero comprobamos que el número del usuario se mantenga entre ese rango con un condicional (si el número introducido no está entre el 1 y el 50, se sale del script mostrando un mensaje). Cuando la condición es falsa, procedemos a ejecutar el bucle, donde le pedimos que vaya mostrando los números que NO coinciden con el introducido por el usuario, y sumando 1 al valor de la variable var hasta que coincida con el valor que introdujo el usuario. Entonces nos mostrará en pantalla el número.

La salida será similar a esta, dependiendo del número introducido:

```
Elige un número entre 1 y 50: 8

El número elegido NO es 1

El número elegido NO es 2

El número elegido NO es 3

El número elegido NO es 4

El número elegido NO es 5

El número elegido NO es 6
```

```
El número elegido NO es 7

El número elegido NO es 8

El número elegido es  8
```

Vemos como el bucle realiza el recorrido por el valor de las variables y las va mostrando con el mensaje que le hemos programado. Los usos que podemos darle a esta estructura, como puedes ver, son muchos. De ti depende el saber aprovecharla en tus scripts.

3.3.1 - Until

Esta estructura es muy similar a **while**, como ya hemos mencionado, pero en lugar de ejecutar el bucle mientras que la condición sea verdadera, se ejecuta mientras la condición sea falsa. Coloquialmente hablando, sería como decir "*hasta que la condición no sea la indicada, ejecuta los comandos siguientes*". Podríamos decir que es un **while**, pero a la inversa.

Su sintaxis es exactamente la misma que en **while**:

```
WHILE [ condición ];

DO

  (comando1)

  (comando2)

  (comando3)

   ....

DONE
```

La forma de utilizarlo en un script simple no cambia mucho con la de **while**, salvo en el enfoque ya mencionado:

```
#!/bin/bash

num=0

until [ $num -gt 10 ]; do

    echo "\$num: $num"

    let num=$num+1

done
```

Coloquialmente, el script sería algo como “mientras la variable no sea mayor de 10, suma 1 a su valor y muéstralo en la salida estándar". Este ejemplo es el mismo que hemos utilizado con while, pero modificando el enfoque del condicional. La salida será exactamente igual a la del ejemplo con **while**.

Deberemos tener en cuenta cuando debemos usar uno u otro, según el caso o la necesidad de cada situación concreta. Es muy posible incluso que podamos usarlos indistintamente simplemente cambiando algunos parámetros.

3.4 - For

Los bucles for son elementos que nos permiten recorrer una estructura o lista de elementos y ejecutar los comandos que queramos durante un cierto número de veces, asignando valores a la variable principal del bucle tantas veces como elementos existan en la lista. El funcionamiento de for en un BashScript es distinto del funcionamiento tradicional de for en lenguajes como C o Java. La sintaxis básica en BashScript sería la siguiente:

```
FOR variable IN lista

DO
        (comandos)
DONE
```

En lenguaje más coloquial, la estructura de ejemplo en BashScript que acabamos de ver irá asignando a la *variable* el valor de cada uno de los elementos de la *lista* y ejecutará los *comandos* tantas veces como elementos tenga la lista.

En lenguaje C o C++, invocar la estructura for se haría de forma algo más similar al siguiente ejemplo:

```
int vector[10], var;

FOR (var=0; var<10; var++)

{
        vector[var]=var;
}
```

Como vemos, se pueden apreciar diferencias significativas entre la forma de invocar las estructuras en cada lenguaje, pero su uso es prácticamente el mismo en ambos casos. Por ejemplo, si necesitamos listar los ficheros en un directorio junto a un breve texto y la numeración de cada fichero existente en el directorio, el código para ello podría ser:

```
#!/bin/bash

num=0
```

```
for fichero in $(ls .)
do
        let num=$num+1
        echo $num" - Este es el fichero" $fichero
done
```

La salida del Script una vez ejecutado, sería similar a esta:

```
1 - Este es el fichero clavesgpg.txt
2 - Este es el fichero Descargas
3 - Este es el fichero Documentos
4 - Este es el fichero Documents
5 - Este es el fichero helper
6 - Este es el fichero enc.pl
7 - Este es el fichero enc.pl.save
8 - Este es el fichero Escritorio
9 - Este es el fichero examples.desktop
10 - Este es el fichero file1
```

El resultado será totalmente diferente, dependiendo claro está, del contenido del directorio donde estemos ejecutando el script. Naturalmente, es difícil que existan los mismos ficheros en dos máquinas diferentes, pero la salida principal (la numeración y el mensaje único) serán prácticamente los mismos, diferenciándose en los nombres de los ficheros y el número de ellos.

Si queremos ejecutar un bucle for un poco más “de los de antes” podemos hacer uso del comando seq, el cual nos generar una lista:

```
for I in $(seq 1 10)
do
  echo "Estamos en la posición $I de 10"
done
```

3.4.1 - For “C Style”

Otra manera de la que disponemos para usar un bucle for es la siguiente, mas similar al lenguaje C y C++:

```
FOR (( expr1 ; expr2 ; expr3 ))

DO
        lista comandos
DONE
```

En primer lugar, la expresión 1 (expr1) es evaluada según las reglas descritas en el script. La expresión 2 (expr2) se evalúa de forma repetida hasta que se evalúa a cero. Cada vez que la expresión 2 (expr2) se evalúa de otra forma distinta a cero (!=0), la lista de comandos se ejecuta y la tercera expresión (expr3) es evaluada. Si cualquier expresión es omitida, se evalúa a 1. El valor de retorno es el estado de salida del último comando que se ejecuta en la lista. Si cualquiera de las expresiones es inválida, devuelve un estado “false”. Veamoslo en un ejemplo:

```
#!/bin/bash

limite=10

for (( a=1; a <= limite ; a++ ))

do

echo -n "$a "

done

echo;
```

En este ejemplo, la salida nos devolverá los números listados del 1 al límite (10), de forma incremental:

```
1 2 3 4 5 6 7 8 9 10
```

NOTA

Como hemos usado el modificador -n en el primer echo, el último echo del script lo usamos como retorno de carro para el PROMPT, ya que sin él, éste se mostraría a continuación del último valor. Haz la prueba.

Como puedes observar, las veces que utilizaremos esta forma de invocar un bucle será en función a la necesidad de obtener una salida exclusiva en base a las expresiones que le facilitemos durante la ejecución. Lo más habitual es ejecutarlo de la forma tradicional, ya que controlaremos mejor la salida estándar del bucle durante la ejecución, aunque nunca está de más conocer otros métodos.

4 - Funciones

En programación, una función no es más que un grupo de comandos a los que podemos llamar para ejecutar algo concreto. Las funciones se suelen crear con un propósito concreto para que, pasándole unos valores de referencia durante la llamada, se ejecuten las órdenes pertinentes con el fin de procesar la entrada de datos y que nos devuelva la salida que necesitemos. Esto se logra utilizando **RETURN** en lugar de **EXIT** para salir de la función una vez finalizada. Si solamente necesitamos procesar una variable, nos valdría con capturar el valor de dicha variable al finalizar la función.

La estructura de una función en BashScript es la siguiente:

```
FUNCTION funcion() {

        código

        código

        código

 }
```

Como hemos visto, al crear la función podemos meter tantos comandos y llamadas a otras funciones como necesitemos con el fin de que nuestra función nos devuelva el valor que nos hace falta o realice algún cometido en concreto.

En el siguiente ejemplo, vamos a ver cómo se procesa el valor de la variable $1 (que es la que se pasa al invocar la función) y la propia función se encargará de realizar ciertas comprobaciones:

```
function procesa() {

        if [ -z $1 ]; then

              echo "La variable está vacía."

              return 0

        else

              chain=$1

              chain2=`echo ${chain:0:1} | awk '{print toupper($0)}'`

              chain3=$chain2${chain:1}

              echo "Has escrito "$chain3

        fi

}
```

Esta función tan sencilla lo que hace es convertir la inicial de la cadena que le pasemos a mayúscula. La ejecución de esta función nos devuelve 0 o el valor de la cadena con la inicial en mayúscula, según el caso.

4.1 - Llamadas a funciones

Para que nuestra función sea útil, debemos invocarla desde algún punto del script. La forma de invocarla es bastante simple, pero deberemos tener en cuenta si la llamada se hace esperando un parámetro o si se solicitará el valor desde dentro de la propia función. Si vemos que en el interior de la función, se trabaja con las variables genéricas $?, $1-$9, etc..., entonces será necesario invocar a la función con los parámetros necesarios. En caso de no hacerlo así, el script debería avisarnos del error.

Cuando la función sea la encargada de solicitar la introducción de datos con el fin de procesarlos, no será necesario invocarla con valores de referencia ni parámetros de ningún tipo.

4.2 - Ejemplos de funciones

En el primer ejemplo, hemos visto que la función devolvía una palabra cuya inicial no fuera mayúscula. Aquí podemos ver la función integrada en el script, donde se solicita que el usuario introduzca una palabra para luego, invocar a la función junto al parámetro requerido, donde será procesado:

```
#!/bin/bash

function procesa() {

        if [ -z $1 ]; then

                echo "La variable está vacía."

                return 0

        else

                chain=$1

                chain2=`echo ${chain:0:1} | awk '{print toupper($0)}'`

                chain3=$chain2${chain:1}

                echo "Has escrito "$chain3

        fi

}

read -p "Escribe alguna palabra... " word

procesa $word
```

La salida de la función nos devolverá la palabra procesada al ejecutar el script:

```
Escribe alguna palabra... programación

Programación
```

Como se ha usado $1 en lugar de $*, la función solo procesará la palabra introducida (cadena) hasta el primer espacio. A partir de dicho espacio, el resto será ignorado. Crea tu propia función y realiza diversas pruebas hasta que tengas claro el concepto.

5 - Arrays

Un array es un medio de guardar un conjunto de objetos de la misma clase. La forma de acceder a cada elemento individual del array es mediante un número entero denominado índice. El índice del primer elemento es 0 (cero) y *n-1* es el índice del último elemento, siendo *n*, la dimensión del array. La estructura de un valor cualquiera del array en BashScript es la siguiente:

```
nombre[índice]=valor
```

Por ejemplo:

```
distro[0]=Debian

distro[1]=Ubuntu

distro[2]=Arch

distro[3]=Puppy
```

Si necesitamos declarar varios valores dentro de un array, podemos hacerlo de la siguiente manera:

```
nombre_array=(valor1 valor2 valor3 valor4)
```

...o, de manera más correcta:

```
declare -a nombre_array=(valor1 valor2 valor3 valor4)
```

Para acceder al valor del primer índice en este array, deberemos seleccionarlo así:

```
nombre_array[0]
```

Veámoslo con un ejemplo sencillo usando valores reales:

```
#!/bin/bash

array=(bash ksh csh)
elem=${#array[*]}  # Así calculamos el total de elementos del array
echo "El array tiene $elem elementos. Estos son:"
i=0
while [ $i -lt $elem ]; do
```

```
echo "$i: ${array[$i]}"
let i++
done
```

En el ejemplo, inicializamos el array con tres valores y calculamos el tamaño del array, almacenándolo en la variable len. Luego los mostramos, usando un bucle para ir rellenando cada línea de la salida:

```
El array tiene 3 elementos. Estos son:

0: bash
1: ksh
2: csh
```

Si necesitáramos que algún elemento tuviera un espacio en medio del valor, lo deberíamos declarar usando las comillas simples, por ejemplo:

```
array=('valor 1' 'valor 2' 'valor 3' 'valor 4')
```

5.1 - Operaciones simples con Arrays

Vamos a realizar unas sencillas operaciones con ayuda del ejemplo anterior:

```
echo "Agregando \"zsh\" al final del array"
array=( "${array[@]}" "zsh" )
echo "Listando todos los elementos del array"
echo "${array[@]}"
echo "Listando todos los elementos del array"
echo "${array[*]}" # Otra manera
echo "Borrando \"ksh\""
unset array[1] # Igual que hacer array[1]=
echo "Ahora el array consta de:"
echo "${array[@]}"
echo "Longitud del tercer elemento del array"
len1=${#array[2]}; echo $len1 " letras"
echo "Borrando el array completamente"
unset array
echo "El contenido del array es: ${array[@]}" # Array vacio
```

La salida de estas operaciones sería esta:

```
El array tiene 3 elementos. Estos son:
0: bash
1: ksh
2: csh
Agregando "zsh" al final del array
Listando todos los elementos del array
bash ksh csh zsh
Listando todos los elementos del array
```

```
bash ksh csh zsh
Borrando "ksh"
Ahora el array consta de:
bash csh zsh
Longitud del tercer elemento del array
3
 letras
Borrando el array completamente
El contenido del array es:
```

Como puedes observar, hemos ido realizando diversas operaciones sobre los diferentes valores del array como por ejemplo:

Agregar un valor al final del array → array=("${array[@]}" "zsh")

Listar los elementos del array → echo "${array[@]}"

Otra forma de listar los elementos → echo "${array[*]}"

Borrar un valor del array (de 2 formas) → unset array[1] / array[1]=

Averiguar la longitud de la cadena del tercer valor del array → len1=${#array[2]}; echo $len1

Borrado del array → unset array

La forma de realizar las diversas operaciones, una vez comprendidas, nos será tremendamente útil para realizar cualquier operación donde se vean implicados múltiples valores en una lista.

Existen algunos métodos más de acceder e interactuar con los valores de un array, pero te retamos a que las investigues por ti mismo para asentar los conocimientos básicos de este curso.

6 - Script de ejemplo

Vamos a analizar brevemente un script de ejemplo, donde podemos ver muchos de los elementos que hemos ido aprendiendo durante el curso. El script está comentado para su total comprensión:

```
#!/bin/bash

#

# SCRIPT DE DESARROLLO DE CONCEPTOS BÁSICOS

#

# Limpiamos la pantalla.

clear

typeset -i num # Se establece el tipo de variable de num a numérica y
se le dá valor 0

# BLOQUE DE DECLARACIÓN DE FUNCIONES

# Creamos la funcion para elegir y comprobar el numero elegido

function eligenum() {

read -p "Elige un número del 1 al 10: " num # Capturamos el numero en
la variable num

if [ -z $num ] ; # Si la variable está vacía...

       then

       echo "No has introducido ningún número." # Mostramos un mensaje
de error

       echo "Saliendo del script..."

       echo ""

       exit 1 # Salimos del script y del bucle

else # Si la variable es un número...

       if [[ $num -gt "10" || $num -lt "1" ]] ;

# Si el valor de la variable num es mayor que 10 o menor que 1...

       then

              echo "El número elegido ("$num") no es válido."

# Se muestra mensaje de error

              echo "El rango es entre 1 y 10."
```

```
            echo ""

       else # Y si no, ya tenemos nuestro número, así que salimos de
la función

            control="1"

# Cambiamos el valor de la variable de control para interrumpir el
bucle

       fi

fi

}

# Funcion para convertir la inicial en mayúscula del nombre de usuario

function mayus() {

       if [ -z "$1" ];  # Si está vacío...

            then

            echo "No has introducido el apellido"

# Mensaje de error. Condición solo para el apellido. El nombre siempre
existe

            chain3=$NULL

# Valor de variable vacío para que no muestre nada en esa variable

       else # Si no...

            chain=$1  # Obtenemos el nombre de usuario como cadena
por argumento

            chain2=`echo ${chain:0:1} | awk '{print toupper($0)}'`

# Obtenemos la primera letra y la pasamos a mayuscula

            chain3=$chain2${chain:1} # Concatenamos la inicial en
mayusculas y el resto del nombre

       fi

}

# INICIO DEL SCRIPT Y FIN DEL BLOQUE DE DECLARACIÓN DE FUNCIONES

# Capturamos el valor de las variables de nombre y apellido que
introduzca

# el usuario.

echo "Introduce tu nombre: "

read nombre
```

```
# Comprobamos que la variable no esté vacia, en cuyo caso,
establecemos un nombre

# a partir del usuario que ejecuta el script

if [ -z "$nombre" ] ;

       then

       mayus $(whoami)

# Invocamos a la función junto al nombre de usuario del sistema como
argumento

       nombre=$chain3

# Asignamos el valor a la variable nombre con el resultado de la
función invocada

       echo "Seleccionando el nombre de usuario del sistema..."

       echo "----------------------------------"

       echo "Hola "$nombre"."

       echo "----------------------------------"

       echo ""

else

       echo "Introduce tu apellido: "

# Si tenemos un nombre correcto, solicitamos el apellido

       read apellido

       mayus $nombre

# Invocamos la función mayus() pasandole como argumento el nombre

       nombre=$chain3

# Obtenemos el resultado de la función y la pasamos a la variable
nombre

       mayus $apellido

# Invocamos la función mayus() pasandole como argumento el apellido

       apellido=$chain3

# Obtenemos el resultado de la función y la pasamos a la variable
apellido

       echo "----------------------------------"

       if [ -z $apellido ]; then # Si la variable apellido está vacía

              echo "Hola "$nombre"."  # Mostramos solo el nombre
```

```
        else

              echo "Hola "$nombre $apellido"." # Y si contiene algo, lo
mostramos todo

        fi

        echo "----------------------------------"

        echo ""

fi

# Inicio de los bucles que solicitan los números al usuario

# Inicializamos la variable de control para el bucle

control="0"

while [ $control -eq "0" ] ;

# Mientras que el valor de la variable de control sea 0, se ejecutará
la funcion

# invocada de forma indefinida

do

        eligenum

done

num1=$num

# Capturamos el valor del número al salir del bucle en la variable
num1 y lo mostramos

echo "El primer número que has elegido es el: "$num1

echo ""

echo "Vamos a por el segundo número..."

echo ""

#Volvemos a poner a 0 la variable de control para iniciar el segundo
bucle

control="0"

while [ $control -eq "0" ] ;

# Mientras que el valor de la variable de control sea 0, se ejecutará
la funcion

# invocada de forma indefinida

do

        eligenum
```

```
done

num2=$num

# Capturamos el valor del número al salir del bucle en la variable
num2 y lo mostramos

echo "El segundo número que has elegido es el: "$num2

echo ""

echo "Ya tenemos los dos números!!"

echo ""

# Mostramos elección de operación a realizar con los dos números
obtenidos

echo "Elige lo que quieres hacer con los dos números elegidos, "$num1"
y "$num2"..."

echo "     1. Sumar"

echo "     2. Restar"

echo "     3. Multiplicar"

echo "     4. Dividir"

echo ""

read -p "Has elegido la opción: " opc

# Capturamos la opción del usuario en la variable opc

echo ""

# Realizamos las operaciones necesarias y asignamos el valor a las
variables

# correspondientes en el propio CASE

case $opc in

       1)

       let suma=$num1+$num2

       echo "El resultado de sumar "$num1" y "$num2" es "$suma

        ;;

       2)

       let resta=$num1-$num2

       echo "El resultado de restar "$num1" y "$num2" es "$resta

        ;;

       3)
```

```
        let multi=$num1*$num2

        echo "El resultado de multiplicar "$num1" y "$num2" es "$multi

         ;;

        4)

        let divi=$num1/$num2

        res=$(echo $num1/$num2 | bc -l) # Calculamos los decimales de
la división

        resto=${res:0:4} # Mostramos únicamente 3 decimales de la
división

        echo "El resultado de dividir "$num1" y "$num2" es "$divi$resto

         ;;

        *)

        echo "La opción introducida no es correcta."

        echo "Se interrumpe la ejecución del script."

         ;;

esac

echo ""

echo "Hasta la vista!"

echo "Prueba de concepto de Bash Script finalizada."

echo ""
```

Test Final

Responda a las siguientes preguntas tipo test (pueden existir preguntas con varias respuestas):

1. ¿Cual de las siguientes palabras es un operador aritmético?

- ○ a)

  pet

- ○ b)

  met

- ○ c)

  bet

- ◉ d)

  let

2. ¿Cual de los siguientes comandos hará que la salida de un programa se escriba en un fichero de texto?

- ○ a)

  touch fichero.txt

- ○ b)

  chmod +x fichero.txt

- ○ c)

  ls -l > ls-l.txt

- ○ d)

  grep fichero.txt

3. ¿Qué hace la orden var++?

- ○ a)

  Suma dos enteros al valor de la variable.

- ○ b)

  Crea un bucle.

- ○ c)

  Vacía la variable.

- ○ d)

  Procesa el valor y después se aumenta su valor en uno.

4. ¿Como se debe iniciar un script escrito para Bash?

- ○ a)

  #!/bash/sh

- ○ b)

  #!/bang/bash

- ○ c)

  #/shell/bash

- ◉ d)

  #!/bin/bash

5. ¿Cual es la opción de read que nos permite introducir texto legible por el usuario?

- ○ a)

  -p

- ○ b)

  -s

- c)

  -w

- d)

  -text

6. ¿Como podemos redirigir la salida estándar (stdout) al error estándar (stderr) en este comando?

- a)

  grep da * 1>&2

- b)

  grep da * >>$2

- c)

  grep da * #!&2

- d)

  grep da * 2<&1

7. ¿Como podemos trabajar con una variable introducida por referencia en la ejecución del script?

- a)

  Por medio de un "echo" al invocar una función

- b)

  Por medio de la orden read.

- c)

  Por medio de las variables intrínsecas.

- d)

Por medio de una condicional.

8. ¿Para qué utilizamos la "identación" de código?

- a)

  Para identificar las variables.

- b)

  Para agrupar los bucles por un lado y los condicionales por el otro.

- c)

  Para agrupar mejor todas las estructuras y poder identificar el código más fácilmente.

- d)

  Para nada. No sirve nada más que para perder tiempo sangrando código.

9. ¿Cómo podemos mostrar el valor de una variable en tiempo de ejecución?

- a)

  let $variable

- b)

  grep $variable

- c)

  echo &variable

- d)

  echo $variable

10. Queremos filtrar todos los procesos en ejecución por nuestro nombre de usuario. ¿Con qué comando podríamos hacerlo?

- a)

  ps aux | less $USER

- b)

  ps aux | more $USER

- c)

  grep $USER < /dev/null

- d)

  ps aux | grep $USER

11. ¿Cual es la estructura básica de una estructura condicional IF en BashScript?

- a)

  IF-ELSE-FI

- b)

  IF-ELSE-END IF

- c)

  IF-FIN

- d)

  IF-ELIF-FI

12. ¿Cuál es la función de una estructura condicional IF?

- a)

  Evaluar una condición

- b)

  Asignar una variable booleana

- c)

  Ejecutar una condición

- ○ d)

  Ejecutar un código en caso de que se cumpla cierta condición

13. ¿A qué corresponde el descriptor "stdin"?

- ○ a)

  A la salida estándar

- ◉ b)

  A la entrada estándar

- ○ c)

  A la entrada no estándar

- ○ d)

  A la salida trasera

14. ¿Como podemos salir de una estructura While mientras se ejecuta, por medio de código?

- ○ a)

  Por medio de un return en alguna parte del código.

- ○ b)

  Por medio de la interruppción del flujo de datos con Ctrl+C.

- ○ c)

  No se puede salir. Es un bucle infinito.

- ◉ d)

  Por medio de la no consecución de la condición programada en la declaración de estructura.

15. ¿Para qué podemos utilizar la sentencia ELIF dentro del condicional IF?

- a)

  Para invocar a una función externa

- b)

  Para ahorrar el anidado de sentencias IF

- c)

  Para salir del bucle

- d)

  Para agregar una condicional adicional a la función invocada

16. ¿Como podemos conseguir el mismo resultado que con la estructura if ["$cadena1" == "$cadena2"]?

- a)

  if [test $cadena1 == $cadena2]

- b)

  if [[$cadena1 -eq $cadena2]]

- c)

  if test "$cadena1" = "$cadena2"

- d)

  if [[test -eq $cadena1 && $cadena2]]

17. ¿Como podemos redirigir la salida de un comando a la entrada del siguiente para ser procesada?

- a)

  Con una tubería

- b)

Con un redirector

- c)

Con un director

- d)

Con un fichero intermedio

18. ¿Qué es una variable?

- a)

Un elemento que puede sufrir variaciones en su valor en tiempo de ejecución, por diversos medios.

- b)

Un elemento que se puede llamar en tiempo de ejecución para que realice una función concreta con unos parámetros dados.

- c)

Una cadena de caracteres convertible pero que nunca cambia de valor una vez declarada.

- d)

Un número.

19. ¿Como podemos cambiar los permisos de ejecución de un fichero de script para poder lanzarlo?

- a)

change +x fichero.sh

- b)

chmore +x ./fichero.sh

- c)

chown +x fichero.sh

- d)

  chmod +x fichero.sh

20. ¿Qué es una función?

- a)

  Es una estructura donde se almacenan variables que se ejecutan entre si.

b)

  Es grupo de comandos a los que podemos llamar en tiempo de ejecución para obtener algo concreto por medio de su codigo interno.

- c)

  Es una forma de ejecutar operaciones simples un numero determinado de veces.

- d)

  Es un compuesto de varias estructuras de control unidas por el valor de sus variables globales.

1. ¿Cual de las siguientes palabras es un operador aritmético?

- ○ a)

  pet

- ○ b)

  met

- ○ c)

  bet

- ◉ d)

  let

Explicación
La respuesta correcta es la D.
2. ¿Cual de los siguientes comandos hará que la salida de un programa se escriba en un fichero de texto?

- ○ a)

  touch fichero.txt

- ○ b)

  chmod +x fichero.txt

- ◉ c)

  ls -l > ls-l.txt

- ○ d)

  grep fichero.txt

Explicación
La respuesta correcta es la C.
3. ¿Qué hace la orden var++?

- ○ a)

Suma dos enteros al valor de la variable.

- b)

Crea un bucle.

- c)

Vacía la variable.

- d)

Procesa el valor y después se aumenta su valor en uno.

Explicación
La respuesta correcta es la D. El operador ++ al final de una variable, procesa su valor y le suma un entero.

4. ¿Como se debe iniciar un script escrito para Bash?

- a)

#!/bash/sh

- b)

#!/bang/bash

- c)

#/shell/bash

- d)

#!/bin/bash

Explicación
La respuesta correcta es la D. Usar #!/bin/bash en la cabecera del fichero es la forma correcta de llamar a la ruta del intérprete e identificar así el fichero como un script para bash.

5. ¿Cual es la opción de read que nos permite introducir texto legible por el usuario?

- a)

-p

- b)

  -s

- c)

  -w

- d)

  -text

Explicación
La respuesta correcta es la A. La opción -p permite insertar texto visible a la hora de capturar el valor en la variable introducido por el usuario.

6. ¿Como podemos redirigir la salida estándar (stdout) al error estándar (stderr) en este comando?

- a)

  grep da * 1>&2

- b)

  grep da * >>$2

- c)

  grep da * #!&2

- d)

  grep da * 2<&1

Explicación
La respuesta correcta es la A

7. ¿Como podemos trabajar con una variable introducida por referencia en la ejecución del script?

- a)

  Por medio de un "echo" al invocar una función

- b)

Por medio de la orden read.

- c)

  Por medio de las variables intrínsecas.

- d)

  Por medio de una condicional.

Explicación
La respuesta correcta es la C, ya que son las variables intrínsecas las que nos permiten usar los valores que lanzamos al invocar el script o durante su ejecución.

8. ¿Para qué utilizamos la "identación" de código?

- a)

  Para identificar las variables.

- b)

  Para agrupar los bucles por un lado y los condicionales por el otro.

- c)

  Para agrupar mejor todas las estructuras y poder identificar el código más fácilmente.

- d)

  Para nada. No sirve nada más que para perder tiempo sangrando código.

Explicación
La respuesta correcta es la C. El identado nos ayuda a identificar los elementos dentro de nuestro código.

9. ¿Cómo podemos mostrar el valor de una variable en tiempo de ejecución?

- a)

  let $variable

- b)

  grep $variable

- c)

  echo &variable

- d)

  echo $variable

Explicación
La respuesta correcta es la D, ya que con "echo" mostramos lo que necesitemos y con $ obtenemos el valor de la variable.

10. Queremos filtrar todos los procesos en ejecución por nuestro nombre de usuario. ¿Con qué comando podríamos hacerlo?

- a)

  ps aux | less $USER

- b)

  ps aux | more $USER

- c)

  grep $USER < /dev/null

- d)

  ps aux | grep $USER

Explicación
La respuesta correcta es la D, ya que con el comando GREP filtramos las líneas de salida de "ps aux" por el valor de la variable del sistema $USER, que es nuestro usuario actual.

11. ¿Cual es la estructura básica de una estructura condicional IF en BashScript?

- a)

  IF-ELSE-FI

- b)

  IF-ELSE-END IF

- c)

IF-FIN

- d)

IF-ELIF-FI

Explicación
La respuesta correcta es la D por definición, al ser considerada la estructura más básica para los condicionales IF.

12. ¿Cuál es la función de una estructura condicional IF?

- a)

Evaluar una condición

- b)

Asignar una variable booleana

- c)

Ejecutar una condición

- d)

Ejecutar un código en caso de que se cumpla cierta condición

Explicación
La respuesta correcta es la D por definición.

13. ¿A qué corresponde el descriptor "stdin"?

- a)

A la salida estándar

- b)

A la entrada estándar

- c)

A la entrada no estándar

- d)

A la salida trasera

Explicación
La respuesta correcta es la B. "stdin" es el descriptor de la entrada estándar al sistema.
14. ¿Como podemos salir de una estructura While mientras se ejecuta, por medio de código?

- a)

  Por medio de un return en alguna parte del código.

- b)

  Por medio de la interruppción del flujo de datos con Ctrl+C.

- c)

  No se puede salir. Es un bucle infinito.

- d)

  Por medio de la no consecución de la condición programada en la declaración de estructura.

Explicación
La respuesta correcta es la D. Cuando deja de darse la condición declarada, se sale del bucle. Es decir, mientras sea verdadera, se ejecutará.
15. ¿Para qué podemos utilizar la sentencia ELIF dentro del condicional IF?

- a)

  Para invocar a una función externa

- b)

  Para ahorrar el anidado de sentencias IF

- c)

  Para salir del bucle

- d)

  Para agregar una condicional adicional a la función invocada

Explicación
La respuesta correcta es la B por definición.
16. ¿Como podemos conseguir el mismo resultado que con la estructura if ["$cadena1" == "$cadena2"]?

- a)

  if [test $cadena1 == $cadena2]

- b)

  if [[$cadena1 -eq $cadena2]]

- c)

  if test "$cadena1" = "$cadena2"

- d)

  if [[test -eq $cadena1 && $cadena2]]

Explicación
La respuesta correcta es la C. Son estructuras que realizan la misma función ya que los corchetes son un atajo al programa test, que es el que ejecuta la acción de comparación.
17. ¿Como podemos redirigir la salida de un comando a la entrada del siguiente para ser procesada?

- a)

  Con una tubería

- b)

  Con un redirector

- c)

  Con un director

- d)

  Con un fichero intermedio

Explicación
La respuesta correcta es la A
18. ¿Qué es una variable?

- a)

  Un elemento que puede sufrir variaciones en su valor en tiempo de ejecución, por diversos medios.

- b)

  Un elemento que se puede llamar en tiempo de ejecución para que realice una función concreta con unos parámetros dados.

- c)

  Una cadena de caracteres convertible pero que nunca cambia de valor una vez declarada.

- d)

  Un número.

Explicación
La respuesta correcta es la A por definición.
19. ¿Como podemos cambiar los permisos de ejecución de un fichero de script para poder lanzarlo?

- a)

  change +x fichero.sh

- b)

  chmore +x ./fichero.sh

- c)

  chown +x fichero.sh

- d)

  chmod +x fichero.sh

Explicación
La respuesta correcta es la D, ya que el comando chmod es el que nos permitirá cambiarle los permisos a cualquier fichero o directorio.
20. ¿Qué es una función?

- ○ a)

  Es una estructura donde se almacenan variables que se ejecutan entre si.

- ○ b)

  Es grupo de comandos a los que podemos llamar en tiempo de ejecución para obtener algo concreto por medio de su codigo interno.

- ○ c)

  Es una forma de ejecutar operaciones simples un numero determinado de veces.

- ◉ d)

  Es un compuesto de varias estructuras de control unidas por el valor de sus variables globales.

Explicación
La respuesta correcta es la B por definición.